

```
In [ ]: import marimo as mo
import random
from ortools.sat.python import cp_model
```

Trabalho Prático: Sudoku Genérico como CSP

1. Introdução

Neste trabalho construímos um gerador/resolvedor de Sudoku $n^2 \times n^2$, com n parametrizável ($n = 3$ corresponde ao Sudoku clássico 9×9).

O Sudoku é tratado como um **Problema de Satisfação de Restrições (CSP)**. A regra é sempre a mesma: um grupo de células tem de ter valores todos diferentes. O que muda entre linhas, colunas e blocos é apenas que células pertencem ao grupo. Por isso o trabalho assenta numa única abstração, a classe `box`, a partir da qual se constrói tudo o resto.

Para resolver o CSP usamos o **CP-SAT** da biblioteca **Google OR-Tools**, por duas razões: permite ter uma variável inteira por célula, com valores em $[1, n^2]$, e já inclui a restrição "todos diferentes" (`AddAllDifferent`), que é exatamente a regra do Sudoku.

2. `box`: grupo genérico de células (R1)

A classe `box` representa um conjunto qualquer de células da grelha às quais se aplica a restrição "todos diferentes". Não sabe nada sobre linhas, colunas, blocos ou Sudoku.

Estrutura de dados. As células são guardadas num dicionário `(linha, coluna) → valor`, em que `None` indica uma célula livre e um inteiro indica uma célula fixa a esse valor. Escolhemos um dicionário por duas razões. Primeiro, permite distinguir três casos: a célula não pertence ao grupo (a chave não existe), pertence e está livre (`None`), ou pertence e está fixa (um inteiro). Segundo, um grupo tem poucas células em relação à grelha toda (por exemplo, 9 em 81 quando $n = 3$), e assim só guardamos as que pertencem ao grupo.

A classe tem:

- o construtor `box(celulas=None, n=3)`, que recebe opcionalmente um dicionário inicial de células e o n da grelha. O n é um parâmetro, com valor por omissão 3 (Sudoku clássico), para que nada fique fixo a 9×9 ;

- o método `add(i, j, val=None)`, que acrescenta a célula (i, j) ao grupo e levanta `ValueError` se as coordenadas estiverem fora da grelha ou o valor fora de $[1, n^2]$. Se a célula já existir no grupo, um valor novo substitui o anterior, mas acrescentá-la sem valor não altera o que lá estava;
- o método `matriz()`, que devolve o grupo como matriz $n^2 \times n^2$, com zeros nas células que não pertencem ao grupo ou não estão fixas.

```
In [ ]: #R1: Grupo genérico de células com a restrição "todos diferentes"

class box: # vai permitir criar vários grupos

    # Construtor: guarda o n e o dicionário (linha, coluna) -> valor ou None
    def __init__(self, celulas=None, n=3):
        self.n = n
        self.celulas = {}
        if celulas is not None:
            for (i,j) in celulas: #percorre as chaves do dicionario que são
                passadas
                self.add(i,j,celulas[(i,j)]) #adiciona o valor corresponde
                à chave

    # Add: Acrescenta a célula (i,j) ao grupo, depois de verificar as
    coordenadas e o valor
    def add(self, i, j, val=None):
        N = self.n * self.n

        #rejeita coordenadas fora da grelhas
        if i < 0 or i >= N or j < 0 or j >=N:
            raise ValueError("coordenadas fora da grelha")

        #rejeita valores fora do intervalo[1, n^2]
        if val is not None:
            if val < 1 or val > N:
                raise ValueError("valor fora do intervalo")

        #acrescentar uma célula sem valor, que já exista, não altera nada
        if val is None and (i,j) in self.celulas:
            return

        self.celulas[(i,j)] = val

    # Matriz: devolve o grupo como matriz n^2 x n^2 (0 = célula livre ou
    fora do grupo)
    def matriz(self):
        N = self.n * self.n

        #faz uma matriz só de zeros
        m = []
        for i in range(N):
            linha = []
            for j in range(N):
                linha.append(0)
            m.append(linha)
```

```

        #percorre as células do grupo, nas que têm valor fixo escreve esse
valor
        for (i,j) in self.celulas:
            if self.celulas[(i,j)] is not None:
                m[i][j] = self.celulas[(i,j)]
        return m

```

3. `cube` e `path`: blocos, linhas e colunas (R2, R3)

`cube` e `path` são duas especializações de `box`. Herdam os métodos `add` e `matriz` e só definem o construtor, que decide que células pertencem ao grupo. Todas as células são acrescentadas livres (sem valor fixo).

- `cube(i, j, n=3)` representa o bloco $n \times n$ de índices (i, j) , com $0 \leq i, j < n$, cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$.
- `path(inicio, fim, n=3)` representa o troço reto, horizontal ou vertical, entre as células `inicio` e `fim`, inclusive. Funciona nos dois sentidos, porque o grupo é o mesmo conjunto de células quer se vá de `inicio` para `fim` quer ao contrário.

Como as células são acrescentadas com o `add` do `box`, a verificação das coordenadas é reaproveitada: um bloco ou um troço fora da grelha levanta `ValueError`.

```

In [ ]: # R2: bloco n x n cujo canto superior esquerdo é a célula (i*n, j*n)
class cube(box):

    # Construtor: cria um grupo vazio e acrescenta as n*n células do bloco
    (i, j)
    def __init__(self, i, j, n=3): #i e j indices do bloco(linha e coluna
do bloco)
        super().__init__(None,n) #cria o grupo vazio
        for a in range(n):
            for b in range(n):
                self.add(i * n + a, j * n + b)

# R3: troço reto (horizontal ou vertical) entre as células inicio e fim,
inclusive
class path(box):

    # Construtor: cria um grupo vazio e acrescenta as células entre inicio
e fim
    def __init__(self, inicio, fim, n=3):
        super().__init__(None,n)
        (i1, j1) = inicio
        (i2, j2) = fim

        # troço horizontal: mesma linha, percorre as colunas

```

```

        if i1 == i2:
            for j in range(min(j1, j2), max(j1, j2) + 1):
                self.add(i1, j)

        # troço vertical: mesma coluna, percorre as linhas
        elif j1 == j2:
            for i in range(min(i1, i2), max(i1, i2) + 1):
                self.add(i, j1)

        # rejeita troços que não são retos
        else:
            raise ValueError("início e fim não estão na mesma linha nem na
mesma coluna")

```

```

In [ ]: # Exemplos de utilização de box, cube e path/ Gerado por LLM

# box: uma célula fixa a 5 não perde o valor se for acrescentada outra vez
sem valor
_b = box()
_b.add(0, 0, 5)
_b.add(0, 0)
print(_b.celulas)

# cube: bloco do meio de um Sudoku 9x9 (linhas 3 a 5, colunas 3 a 5)
print(cube(1, 1).celulas)

# path: linha 0 inteira, dada do fim para o início
print(path((0, 8), (0, 0)).celulas)

# cube com n=2: bloco de baixo à esquerda de uma grelha 4x4
print(cube(1, 0, n=2).celulas)

```

```

{(0, 0): 5}
{(3, 3): None, (3, 4): None, (3, 5): None, (4, 3): None, (4, 4): None, (4,
5): None, (5, 3): None, (5, 4): None, (5, 5): None}
{(0, 0): None, (0, 1): None, (0, 2): None, (0, 3): None, (0, 4): None, (0,
5): None, (0, 6): None, (0, 7): None, (0, 8): None}
{(2, 0): None, (2, 1): None, (3, 0): None, (3, 1): None}

```

4. Geração aleatória de pistas (R4)

A função `pistas(k=None, n=3)` devolve um `box` com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor aleatório em $[1, n^2]$. Não foi precisa nenhuma classe nova: o resultado é apenas um `box`, preenchido com o `add`.

- Se k não for dado, usa-se $k = n$ (3 pistas no Sudoku clássico).
- Se k for negativo ou maior do que o número de células da grelha, a função levanta `ValueError`.
- As células são sorteadas com `random.sample` sobre a lista de todas as células da grelha, o que garante k células diferentes.
- O valor de cada célula é sorteado com `random.randint`, de forma independente das outras, por isso podem se repetir.

Como os valores são sorteados de forma independente, duas pistas podem ficar com o mesmo valor. O modelo trata o grupo das pistas como qualquer outro grupo (todos diferentes), por isso nesse caso o puzzle fica impossível. Essa situação é tratada na secção 5(R5,R6).

```
In [ ]: # R4: devolve um box com k células escolhidas ao acaso, cada uma fixa a um
        # valor ao acaso
        def pistas(k=None, n=3):
            # tamanho da grelha
            N = n*n

            # se o k não for dado, fica igual a n
            if k is None:
                k = n
            # rejeita um k negativo ou maior do que o número de células
            if k < 0 or k > N * N:
                raise ValueError("k tem de estar entre 0 e o número de células da
grelha")

            # lista com todas as células (i, j) da grelha
            lista = []
            for i in range(N):
                for j in range(N):
                    lista.append((i,j))

            # sorteia k células diferentes dessa lista
            escolhidas = random.sample(lista, k)

            # cria um grupo vazio com este n
            grupo = box(n=n)

            # acrescenta cada célula sorteada ao grupo, com um valor ao acaso entre
1 e N
            for (i,j) in escolhidas:
                grupo.add(i, j, random.randint(1, N))

            # devolve o grupo
            return grupo
```

```
In [ ]: # Exemplo: 3 pistas aleatórias numa grelha 9x9
        _g = pistas()
        print(_g.celulas)
        for _linha in _g.matriz():
            print(_linha)
```

```
{(8, 8): 6, (1, 8): 7, (8, 6): 2}
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 7]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 2, 0, 6]
```

5. Modelo CSP e resolução (R5, R6)

A classe `sudoku` é o modelo CSP da grelha $n^2 \times n^2$. Usa o CP-SAT e tem três partes:

- o construtor `sudoku(n=3)`, que cria o modelo e uma variável inteira por célula, cada uma no intervalo $[1, n^2]$. As variáveis ficam num dicionário `x`, com a posição (i, j) como chave;
- o método `junta(*grupos)`, que recebe um número arbitrário de grupos. Para cada grupo, impõe a restrição "todos diferentes" (`AddAllDifferent`) sobre as variáveis das suas células e fixa as células que tiverem valor atribuído. O método só usa o dicionário `celulas` do grupo, por isso não distingue se é um `box`, um `cube`, um `path` ou um grupo de pistas;
- o método `resolve()`, que chama o solver e devolve a grelha preenchida, como lista de listas, ou `None` se o puzzle não tiver solução.

```
In [ ]: # R5: modelo CSP para uma grelha n^2 x n^2, com uma variável inteira por
        célula
        class sudoku:

            #1: (Criar Variveis) Construtor: cria o modelo e as variáveis, uma por
            célula, com valores em [1, n^2]
            def __init__(self, n=3):
                self.n = n
                # cria um modelo vazio do CP-SAT e guarda-o no objeto, para os
                outros métodos o usarem
                # aqui ficam guardadas as variáveis(mais tarde restrições)
                self.modelo = cp_model.CpModel()
                # cria dicionário vazio para as variáveis: serve para encontrar
                cada variável pela posição
                self.x = {}
                N = self.n * self.n
                #para cada célula(i,j) da grelha, cria uma variável inteira entre 1
                a N e guarda-a no dicionário, com a chave (i,j)
                for i in range(N):
                    for j in range(N):
                        self.x[(i, j)] = self.modelo.NewIntVar(1, N, "x_%i_%i" %
                        (i, j)) #guarda a variável no dicionário, para encontrar
                        depois a celulas pela posição
```

```

#2: (Guarda restrições) recebe um número arbitrário de grupos; para
cada grupo impõe "todos diferentes"
#e fixa as células que tiverem valor atribuído
def junta(self, *grupos):
    for g in grupos:
        variaveis = [] #cria uma lista vazia para as variáveis das
        células deste grupo
        for (i,j) in g.celulas:
            variaveis.append(self.x[(i,j)])
            # se a célula tiver valor fixo, acrescenta ao modelo a
            restrição
            if g.celulas[(i,j)] is not None:
                self.modelo.Add(self.x[(i,j)] == g.celulas[(i,j)])

        #acrescenta ao modelo a restrição "todos diferentes" sobre a
        lista
        self.modelo.AddAllDifferent(variaveis)

#3: (Resolver) resolve o modelo; devolve a grelha preenchida, ou None
se o puzzle não tiver solução
def resolve(self):
    solver = cp_model.CpSolver() #Cria o solver, o programa que vai
    procurar a solução
    estado = solver.Solve(self.modelo) #manda o solver resolver o
    modelo, devolve um código como correu, que fica guardado em estado

    # há solução: constrói a grelha com o valor de cada variável
    if estado == cp_model.OPTIMAL or estado == cp_model.FEASIBLE: #aqui
    verifica se foi encontrada solução
        N = self.n * self.n
        grelha = []
        for i in range(N):
            linha = []
            for j in range(N): #usar valores para todas as variáveis
                linha.append(solver.Value(self.x[(i, j)]))
            grelha.append(linha)
        return grelha

    # não há solução
    else:
        return None

```

```

In [ ]: # Com solução: só a linha 0 tem a restrição "todos diferentes"
_s = sudoku()
_s.junta(path((0, 0), (0, 8)))
for _linha in _s.resolve():
    print(_linha)

# Sem solução: duas células do mesmo grupo fixas ao mesmo valor
_s2 = sudoku()
_s2.junta(box({(0, 0): 5, (0, 1): 5}))
print(_s2.resolve())

```

```

[9, 8, 7, 6, 5, 4, 3, 2, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
None

```

Sudoku completo (R6)

A função `resolve_sudoku(p, n=3)` monta um Sudoku completo: cria um modelo `sudoku` e junta-lhe todas as linhas e todas as colunas (com `path`), todos os blocos (com `cube`) e o grupo de pistas `p`. Para $n = 3$ são 28 grupos, todos tratados da mesma forma pelo `junta`. A montagem é uma função fora da classe `sudoku` para o modelo continuar genérico: outras variantes de Sudoku fazem-se juntando outros grupos, sem alterar a classe.

Puzzle sem solução. As pistas são sorteadas ao acaso, por isso o puzzle pode não ter solução (por exemplo, quando duas pistas ficam com o mesmo valor). Nesse caso optámos por sortear pistas novas e tentar outra vez, em vez de apenas reportar o insucesso, porque o objetivo é obter um Sudoku resolvido. A função `gera_sudoku(k=None, n=3, tentativas=20)` faz isso e devolve as pistas usadas e a grelha. Cada tentativa cria um modelo novo, porque não se podem retirar restrições de um modelo. O número de tentativas é limitado para o programa nunca ficar preso: se nenhuma tiver solução, a função devolve `None`. É o que acontece sempre que $k > n^2$, porque as pistas têm de ser todas diferentes e só há n^2 valores.

Apresentação. A função `mostra(grelha, n=3)` escreve a grelha em texto, com separadores entre os blocos. Escolhemos texto por ser simples, funcionar para qualquer n e deixar ver os blocos. A mesma função mostra as pistas, a partir do `matriz()` do grupo, em que os zeros são as células por preencher.

```

In [ ]: # R6: monta o Sudoku completo (linhas + colunas + blocos + pistas) e
        resolve-o
def resolve_sudoku(p, n=3):
    # tamanho da grelha
    N = n * n
    # cria um modelo vazio novo para este n(ainda sem restrições)
    s = sudoku(n)
    # junta todas as linhas e todas as colunas
    for i in range(N):
        s.junta(path((i,0), (i,N-1), n = n))
        s.junta(path((0,i), (N-1,i), n = n))

    # para cada bloco (i, j), com i e j de 0 a n-1, junta ao modelo esse
    bloco
    for i in range(n):

```

```

        for j in range(n):
            s.junta(cube(i, j, n=n))
        # junta ao modelo o grupo das pistas
        s.junta(p)

        # resolve e devolve o resultado
        return s.resolve()

# R6: sorteia pistas até obter um puzzle com solução; devolve as pistas e a
grelha
def gera_sudoku(k=None, n=3, tentativas=20):
    for t in range(tentativas):
        p = pistas(k, n)
        grelha = resolve_sudoku(p, n)

        # encontrou um puzzle com solução
        if grelha is not None:
            return p, grelha

    # nenhuma das tentativas teve solução
    return None, None

# R6: mostra a grelha em texto, com separadores entre os blocos
def mostra(grelha, n=3):
    N = n * n
    for i in range(N):
        # linha de traços entre blocos
        if i > 0 and i % n == 0:
            print("-" * (3 * N + 2 * (n - 1)))

        texto = ""
        for j in range(N):
            # barra vertical entre blocos
            if j > 0 and j % n == 0:
                texto = texto + "|"
            texto = texto + "%2i " % grelha[i][j]
        print(texto)

```

```

In [ ]: # Sudoku 9x9 completo: sorteia pistas, resolve e mostra
        _p, _g = gera_sudoku()
        if _g is None:
            print("Não foi encontrado um puzzle com solução")
        else:
            print("Pistas:")
            mostra(_p.matriz())
            print()
            print("Solução:")
            mostra(_g)

```

Pistas:

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		4	0	0
0	0	0		0	0	0		0	5	0

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0

0	0	0		0	0	0		0	0	0
0	0	1		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0

Solução:

4	1	8		5	2	7		3	6	9
5	7	6		3	1	9		4	2	8
3	2	9		4	8	6		1	5	7

6	4	2		1	7	3		9	8	5
1	3	5		2	9	8		7	4	6
9	8	7		6	4	5		2	1	3

2	5	3		9	6	1		8	7	4
8	9	1		7	5	4		6	3	2
7	6	4		8	3	2		5	9	1

6. Testes e validação

Esta secção verifica automaticamente o que o enunciado pede: que a grelha resolvida é uma solução válida, que as pistas se mantêm, que o `add` rejeita células inválidas, e que o fluxo completo funciona com mais do que um valor de n .

6.1 Validação da solução: linhas, colunas, blocos e pistas

A função `valida(grelha, p, n=3)` devolve `True` só se:

- cada linha, cada coluna e cada bloco $n \times n$ tiver exatamente os valores de 1 a n^2 , sem repetições;
- cada célula fixada pelas pistas `p` tiver, na grelha, o valor com que foi fixada.

Para testar um conjunto de células, ordenam-se os seus valores e compara-se o resultado com a lista $[1, \dots, n^2]$. A validação percorre a grelha diretamente pelos índices, sem usar `path` nem `cube`, para não depender do código que está a ser testado.

O teste a seguir à função confirma que ela aceita uma solução correta e rejeita uma grelha estragada de propósito.

```
In [ ]: # Verifica se a grelha é uma solução válida do Sudoku e respeita as pistas p
```

```

def valida(grelha, p, n=3):
    N = n * n
    certo = list(range(1, N + 1))

    # cada linha tem exatamente os valores de 1 a N
    for i in range(N):
        valores = []
        for j in range(N):
            valores.append(grelha[i][j])
        if sorted(valores) != certo:
            return False

    # cada coluna tem exatamente os valores de 1 a N
    for j in range(N):
        valores = []
        for i in range(N):
            valores.append(grelha[i][j])
        if sorted(valores) != certo:
            return False

    # cada bloco n x n tem exatamente os valores de 1 a N
    for bi in range(n):
        for bj in range(n):
            valores = []
            for a in range(n):
                for b in range(n):
                    valores.append(grelha[bi * n + a][bj * n + b])
            if sorted(valores) != certo:
                return False

    # as células das pistas mantêm o valor com que foram fixadas
    for (i, j) in p.celulas:
        if p.celulas[(i, j)] is not None:
            if grelha[i][j] != p.celulas[(i, j)]:
                return False

    # passou em todas as verificações
    return True

```

```

In [ ]: # A valida aceita uma solução correta e rejeita uma grelha estragada
        _p, _g = gera_sudoku()
        print("solução correta é aceite:", valida(_g, _p))

        # estraga a grelha de propósito: a coluna 0 fica com um valor repetido
        _g[0][0] = _g[1][0]
        print("grelha estragada é rejeitada:", not valida(_g, _p))

```

solução correta é aceite: True
grelha estragada é rejeitada: True

6.2 O `add` rejeita coordenadas e valores inválidos

A função `rejeita(i, j, val=None, n=3)` tenta acrescentar uma célula a um `box` vazio e devolve `True` se o `add` levantar `ValueError`. Os testes confirmam

que são rejeitadas coordenadas fora da grelha (acima do limite ou negativas) e valores fora de $[1, n^2]$, e que uma célula válida é aceite. Testa-se também que o `path` rejeita um troço que não é reto.

```
In [ ]: # Tenta acrescentar uma célula a um box vazio; devolve True se o add a
         # rejeitar
def rejeita(i, j, val=None, n=3):
    try:
        box(n=n).add(i, j, val)
        return False
    except ValueError:
        return True
```

```
In [ ]: # 0 add rejeita coordenadas fora da grelha e valores fora de [1, n^2]
print("linha fora da grelha:", rejeita(9, 0))
print("coluna fora da grelha:", rejeita(0, 9))
print("coordenada negativa:", rejeita(-1, 0))
print("valor abaixo de 1:", rejeita(0, 0, 0))
print("valor acima de n^2:", rejeita(0, 0, 10))
print("célula válida é aceite:", not rejeita(0, 0, 5))

# 0 path rejeita um troço que não é reto
try:
    path((0, 0), (1, 1))
    print("troço diagonal é rejeitado:", False)
except ValueError:
    print("troço diagonal é rejeitado:", True)
```

```
linha fora da grelha: True
coluna fora da grelha: True
coordenada negativa: True
valor abaixo de 1: True
valor acima de n^2: True
célula válida é aceite: True
troço diagonal é rejeitado: True
```

6.3 Fluxo completo com $n = 3$ e $n = 2$

A função `fluxo(n)` corre o fluxo completo: sortear pistas, montar linhas, colunas, blocos e pistas, resolver, validar e mostrar a grelha. É corrida com $n = 3$ (Sudoku clássico 9×9) e com $n = 2$ (grelha 4×4), o que confirma que nada está fixo a 9×9 .

```
In [ ]: # Fluxo completo: sortear pistas -> montar -> resolver -> validar ->
         # mostrar
def fluxo(n):
    p, g = gera_sudoku(n=n)
    if g is None:
        print("n =", n, ": não foi encontrado um puzzle com solução")
    else:
        print("n =", n, ": solução válida?", valida(g, p, n))
        mostra(g, n)

fluxo(3)
```

```
print()
fluxo(2)
```

```
n = 3 : solução válida? True
3 6 8 | 1 2 9 | 5 4 7
1 2 7 | 4 5 6 | 8 9 3
4 5 9 | 3 8 7 | 2 1 6
-----
2 1 3 | 7 6 5 | 4 8 9
7 9 4 | 2 1 8 | 3 6 5
5 8 6 | 9 3 4 | 1 7 2
-----
6 7 1 | 5 4 3 | 9 2 8
8 3 2 | 6 9 1 | 7 5 4
9 4 5 | 8 7 2 | 6 3 1

n = 2 : solução válida? True
4 1 | 3 2
3 2 | 4 1
-----
1 3 | 2 4
2 4 | 1 3
```

7. Utilização de LLM

Neste trabalho usámos um LLM como apoio. Serviu para interpretar o enunciado, para construir o notebook passo a passo com a explicação de cada parte, e para rever o código e os textos. O LLM propôs a estrutura e grande parte do código e dos textos explicativos; nós escrevemos e testámos o código no notebook, corrigimos os erros que foram aparecendo e tomámos as decisões de implementação, como a posição do parâmetro n e o tratamento das células repetidas. Conversa utilizada: <https://claude.ai/share/c5ccee84-2c21-472e-baa0-5eefc97f6ad3>